

Lab5 Report

量化

因为 Gemma4-12B 权重本身就爆显存了，我们必须要进行量化。那根据实验文档的提示，我们使用 GPTQ W4A16 进行量化。

整体过程就是按照文档提示来，对角阻尼取 0.01，以 128 列计算 block，64 列作为 group，最后做到 $\Delta NLL = 0.08$ ，已经达到我们的要求了。

后续发现每次任务重新量化速度太慢，为了开发的方便，我们在本地缓存了一份量化后的权重。

Baseline

那量化完之后我们才有 baseline 可言。

我们测试了一个 datasets/performance_small.jsonl，耗时 76.13s，throughput 0.788 token/s。正式测试会在 prefill 阶段 OOM，所以没有可比较数据。

分析发现 `aten::copy_` 时间占比最大，其次是 INT4 elementwise 运算。GEMM 反而不是热点部分。

显存压缩

那我们的首要目标是先得跑起来，压一压显存就好了。

我们的显存，主要由权重、KV cache、激活值构成。

Offloading

首先权重可以进一步压一压，也就是只在需要的时候把对应的权重加载进来，比方说 offloading。

我们按 layer 分一下，并且做 prefetch，在 layer i 计算的时候，在不同 stream 上预取下一层。并且使用 pinned-memory 加速一下。

测试发现，对于短输入，峰值显存从 8.25GB 降到 2.67GB。当然略微变慢了，但是无所谓。

Ring KV

其次，KV cache 可以压一压。我们 sliding attention layer 只关注 1024 长度的 token，那么保存更早的 KV cache 完全没用，所以可以完全无副作用地压缩 KV cache。

实测结果发现 KV allocation 减少 320MB，47.62%。在长输入下，收益会更明显一些。

Paged attention

最后是一个系统层面的优化。类似于操作系统按 page 从磁盘读取文件的概念，我们对我们的显存也按 block 的粒度管理。这样的好处是管理更加灵活，可以随用随取，用完即释放，为以后的进一步优化打下基础。我们这里采用 vLLM 默认的设置，也就是 block 大小取 16。

仅用 paged attention 其实只是帮我们精细化了管理的粒度，真的想要显存降下来还是得配合着来。

根据题目给的输入输出区间上界，我们可以算出每个 layer 需要多少 block，然后只分配这么多 block 就好了。这样就比之前每个 layer 都按最大值分配要好很多。

然而在跑实际完整 10 请求时，还是会报显存不足（但是这个请求还是能跑完，不太懂为什么）

Prefill logits 只算最后一个

进一步研究之前为什么会报显存不足。分配失败的提示是 `1,048,576,000 B` 这么大分配失败，那计算发现这正好是 `2000 prompt * 262144 vocab * 2 bytes`，这说明这是在生成最后的 logits 的过程中发生的 OOM。

但问题是我们要完整的 logits 根本没用，我们只需要最后一个 token 的 logits。

所以我们只算这个 logits，就好了。峰值变为 `6,297,783,808 B`。

算子优化

baseline profile 显示 INT4 位运算与逐元素反量化远比真正的 GEMM 更高。这是因为代码把反量化跟 GEMM 分开写了，产生了额外的通讯与 overhead。

反量化-GEMM 融合

那很自然的想法就是做算子融合（本来反量化得到的结果也就是个临时值，我干嘛不直接用完顺手释放）。

运行完整 10 请求，400.5s，下降了 5.77%。

profile 结果表明逐元素操作不再是主导，热点变成 GEMM 48.53%。

FlashAttention

QK^T 太大了 ($O(L^2)$)。我们使用 flash attention 技术，不仅减小显存占用，同时我记得 flash attention 是 cache-aware 的，所以也提高了速度。

完整 10 请求降到 366.6s，下降了 8.45%。但是峰值显存没有下降，可能是因为峰值不是由激活值导致的。

调度

我们现在还是要 366s，距离 36s 还差很远。不过问题不大，因为我们目前是串行完成所有请求的，接下来一个优化的大头就是做 batch。

continuous batching

基本上跟实验文档里提示的一致，做完之后，又做了简单的去同步达到 111.4s，下降 69%。

W4A16

到这里进行 profile，我们发现真正的 flashattention 只占 0.15%，而 W4A16（我们之前融合得到的算子）占了 59.91%。所以很明显接下来要重点优化这部分。

量化权重重排

最开始权重是按 `[N, K/2]` 排布的，这就导致一个 warp 在读取 N 这一维上相邻的 32 个数据点时，读取的地址可能造成 bank conflict，GPU 无法把这些读取合并。

解决方法是重排为 `[K/64, N/128, 32, 128]` 这样的形状，也就是把 K 切成 64 一组，N 切成 128 一组，计算某个小块的时候，一个 CTA 负责 128 个输出通道，刚好是挨在一起的，一个 warp 去读地址是连续的。

这样做还有一个额外好处，我们 K 正好是按照量化时的 group 大小分组的，每个 K 组正好对应于一组量化参数。

根据 M 特化 tile 形状

当 M 很小的时候，如果使用大 tile 形状，那么很多行都是空算，浪费算力。

当 M 很大的时候，如果使用小 tile 形状，那么 K 维度的数据要被加载很多次，复用率很低。

所以我们针对大、中、小设计不同形状的 tile。128*64*64, 32*128*64, 16*128*64。

最终在设计的 benchmark 上取得了 5-6x 的加速。

进行 profile，W4A16 已经下降到 39%，说明我们的优化是成功的。

减少通讯

刚刚的 profile 同时表明，pinned H2D 占比 45% 已经成为最大热点。这主要是由于 Offloading 造成的。

然而我们分到的 MIG 只有一个 copy engine，所以多开 stream 等等是没啥用的，我们只能减少通讯量。

合并 payload

profile 表明，H2D 次数高达 33280 次，这说明我们所有的小张量都是分开搬运的，非常糟糕。

所以我们合并搬运就好了。

此外 profile 还表明我们有 33311 次 D2H 搬运。为啥我们要把权重搬回来？直接全部去掉。

搬运次数下降到 1536 次，并且 D2H 全部消失，其耗时也从 14.13s 变成 0。

常驻 MLP 权重

我们已经把 overhead 给干掉了，接下来得正儿八经减少一些搬运。

一个自然的想法就是，反正我 offloading 是在做显存跟计算的 tradeoff，那现在我显存挺宽裕的，能不能反过来常驻一部分权重。

分析发现，权重也就 Attention 和 MLP 两部分，其中 MLP 计算更密集，权重读取更频繁，所以常驻它。

然而尝试之后发现，全部 48 层常驻不下，那就常驻 44 层就好了。

profile 表明总搬运量下降了 71%，并且 H2D 时间从 23.55s->10.17s。

共享 RoPE cache

这是小优化。

简单来说，每层 payload 中 RoPE 是完全重复的，我们没必要存多份。改成只存 sliding 和 global 两份给大家复用就好了。

最终的实测结果大概在 31s 左右，完全达到要求，可以提交。

思考题

量化后权重大小计算

整体上权重就这么几个来源：embedding 矩阵、attention 权重、MLP 权重。后两者可以打包在一个 layer 中整体计算。

我们记 $N = 48$ 为 layer 数， $D = 3840$ 为隐藏层维数， $H_q = 16$ 为注意力头数， $H_{l,kv} = 8, H_{g,kv} = 1$ 分别表示滑动窗口、全局注意力层 KV 头数， $D_{l,h} = 256, D_{g,h} = 512$ 为滑动窗口、全局注意力层每个头的维度， $w = 1024$ 为滑动窗口上下文长度， $F = 15360$ 为 FFN 中间层维数， $V = 262144$ 为词表大小。

对于 embedding 矩阵，它同时作为 LM head，大小为 VD ，精度 BF16，占用 2013265920B。

接下来看每个 layer 内部。

attention 部分，每层有 Q, K, V, O 四个投影，并且 sliding 部分每两组 Q 共用一组 K, V，而 global 部分全局共用。并且 global 部分 K/V 共享。

$$P_{slide} = 40D(H_q D_{l,h} + H_{l,kv} D_{l,h} + H_{l,kv} D_{l,h} + H_q D_{l,h}) = 1887436800$$

$$P_{global} = 8D(H_q D_{g,h} + H_{g,kv} D_{g,h} + H_q D_{g,h}) = 519,045,120$$

MLP 部分，每层都一样，因为我们用的是 SwiGLU，所以有三个矩阵。

$$P_{MLP} = N(DF + DF + FD) = 8493465600$$

加起来，总权重数为 10899947520。

这里的权重是 INT4，占 0.5B，然而还有反量化参数。每 64 个参数有一个 FP16（我们使用对称量化，没有 zero），平均占 $\frac{1}{32}$ B。

综上，一个权重平均占 $\frac{17}{32}$ B。

计算得这部分总共是 5790597120B。

两部分加起来，一共 7803863040B。

粗略计算就结束了，然而还有归一化层。

每层 4 个长度为 D 的 RMSNorm scale，每个 attention 有 Q-Norm K-Norm，最终长度为 D 的 RMSNorm，每层一个 layer scalar。

$$\text{总共 } N(4D) + 2(40D_{l,h} + 8D_{g,h}) + D + N = 769840 \text{ 这么多 BF16, 一共 } 1539680B$$

加起来，总共 7,805,402,720B，约 7.269GiB。实际测试结果为 7,805,505,952 B，比理论算出来多 103,232B，基本上可以认为是 padding 对齐导致的。

KV cache

设 batch 大小为 B ，最长长度为 L 。

当使用 Ring KV Cache 时，对于 40 个 sliding layer 只需要保留 $\min(L, W)$ 个 token，8 个 global layer 仍需完整的 L 。

$$M(B, L) = 4B(40H_{l,kv} D_{l,h} \min(L, w) + 8H_{g,kv} D_{g,h}) = 4B(81920 \min(L, 1024) + 4096L)$$

当 $B = 1, L = 2048$ 时， $M = 369098752$ ，与测试结果相差 384B。推测是分配的 length。

2

sparsity 有很多种方法来实现，比方说我们介绍一种最经典的。

我们把 KV cache 分块，比如每 16 个 token 作为一个大块，把 16 个 K 向量压缩成一个低维向量，当然 Q 也可以压一下。

当需要计算一次 attention 时，我们用压缩后的 K/Q 计算注意力分数。因为这是压缩后的，并且是在低维上做，所以快很多。

这相当于是进行一轮粗筛。进行粗筛过后，我们根据粗筛分数挑出 topk 再正常用 FlashAttention 去算。

显然，我这个做得比滑动窗口要精细很多，所以模型性能会上升。但是中间多了很多计算，多了很多显存需求，所以推理效率会下降。

当然还有别的稀疏化方法，比如引入 retrieval memory 等等。

3

这个问题第一眼看有点无厘头，想了一下感觉它可能是这个意思。

我们在做 offloading 的时候，相当于是我们前面若干层都做完，当前一层正在计算的同时，prefetch 下一层的权重。

我们在做分层量化的时候，其实是相当于前面若干层都做完，我们已经尽力让目前的输入矩阵跟未量化的版本一致了，然后做当前这一层的量化，之后再做下一层的。

总之都有这种逐层处理权重的感觉。

区别就很明显了，它们根本就不是一个东西。

分层量化是在推理前离线完成的，它的目的是把原有权重中的冗余信息压缩掉，变成占地更小的信息，同时结果不太受损。然而它还是保留了所有层的完整信息。

offloading 是在推理时在线进行的，它的目的是把推理当前步用不到的权重都丢掉，只保留当前需要的这一小部分。它没有进行压缩。